

Assignment 10: Decision Trees - Logic, Splits, and Pruning

Data Science and Machine Learning Module

December 2025

1 Part 1: Classification - "The Academic vs. The Builder"

In this section, you will use the `new_placement_data.csv`. This dataset contains **CGPA** (Continuous) and **Number of Projects** (Ordinal: 0–5) for 3,000 students.

1.1 Task 1: The Manual Gini Challenge : NO LIBRARIES

Decision Trees use the **Gini Impurity** to decide where to split.

$$Gini(S) = 1 - \sum p_i^2$$

1. Implement a Python function to calculate Gini Impurity.

2. Calculate:

- Initial Gini of the whole dataset.
- Weighted Gini of a split on **CGPA** at 8.5.
- Weighted Gini of a split on **Projects** at 2.5.

Question: Based on Gini reduction, which feature should be the Root Node? Write the code to find out which of the two features is supposed to be the root node.

1.2 Task 2: Visualizing the Decision Flow

Use `DecisionTreeClassifier` and `plot_tree` from `sklearn.tree`.

```
1 from sklearn.tree import DecisionTreeClassifier, plot_tree #library to use
   decision trees and plot the graphs as shown in Figure 1.
2
3 X = df[['feature_1', 'feature_2', ..., 'feature_n']]
4 #x has the features columns
5 clf = DecisionTreeClassifier(max_depth=5, random_state=42)
6 # clf is the variable which makes the tree with parameters as required
7 #max_depth is the number of branching you will have: it is a hyperparameter
8 clf.fit(X, y)
9 # this is how you train the tree on the data
10
11 #if you want to use the hyperparameter alpha, you can use this code instead
12 clf = DecisionTreeClassifier(ccp_alpha=0.1, random_state=42)
13
14
```

```

15 #CODE FOR PLOTTING THE TREE
16 plt.figure(figsize=(15, 8))
17 plot_tree(clf, feature_names=['feature_1', 'feature_2', ..., 'feature_n'],
18           class_names=['class1', 'class2'],
19           filled=True, rounded=True, fontsize=10)
19 plt.title("Decision Tree Logic for Placement")
20 plt.show()
21
22 #For using the data for testing or validation, we can use:
23 y_pred = clf.predict(X_test)
24
25 X_m = [['x1', 'x2']].values
26 y_m = df['y'].values
27
28 # code to plot the actual decisions for 2 dimensions
29 def plot_surface(clf, ax, title):
30     x_min, x_max = X_m[:, 0].min() - 0.5, X_m[:, 0].max() + 0.5
31     y_min, y_max = X_m[:, 1].min() - 0.5, X_m[:, 1].max() + 0.5
32     xx, yy = np.meshgrid(np.arange(x_min, x_max, 0.02), np.arange(y_min, y_max,
33     0.02))
34     Z = clf.predict(np.c_[xx.ravel(), yy.ravel()]).reshape(xx.shape)
35     ax.contourf(xx, yy, Z, alpha=0.3, cmap='RdBu')
36     ax.scatter(X_m[:, 0], X_m[:, 1], c=y_m, s=5, cmap='RdBu', alpha=0.5)
37     ax.set_title(title)
38
39 # 1. Create a single figure and axis
40 fig, ax = plt.subplots(figsize=(8, 6))
41
42 # 2. Call the function for the specific model you want
43 plot_surface(clf, ax, f"Plot")
44
45 # 3. Final display settings
46 plt.tight_layout()
47 plt.show()

```

Listing 1: Some codes and libraries which might be required

1. Train a tree with `max_depth=3`
2. Generate the decision graph using the `plot_tree` function.
3. **Analysis:** Identify the "Builder Path." For students with a CGPA below 8.0, how many projects does the tree determine they need to be "Placed"?

2 Part 2: Regression - The Sine Wave Staircase

Using `regression_tree.csv`, you will observe how trees approximate smooth curves using piecewise constant "steps" by minimizing the **Sum of Squared Residuals (SSR)**.

2.1 Task 1: SSR and Step-Functions

1. Train two models: a **Shallow Tree** (`max_depth=2`) and a **Deep Tree** (`max_depth=10`).
2. Create a single plot showing:
 - The original noisy scatter points.

- The red "staircase" prediction of the shallow tree.
- The green "jagged" prediction of the deep tree.

3 Part 3: The Pruning Sweet Spot

Decision trees are prone to "memorizing" noise. You will use the `moons_tree.csv` dataset to find the optimal balance using **Cost Complexity Pruning** (α).

$$\text{Cost} = \text{Total Gini Error} + \alpha \times (\text{Number of Terminal Nodes})$$

3.1 Task 1: The Alpha Sweep

1. Split your data into 80% training and 20% validation.
2. Iterate through α values in the range $[0, 0.05]$ (use at least 10 values).
3. Plot **Training Accuracy** and **Validation Accuracy** vs. α .

3.2 Task 2: The Decision Surface

1. Identify the optimal α from your plot.
2. Generate a 2-panel plot (2 plots side by side) showing the **Decision Surface** using the `plot_surface` function defined above (`contourf`) for:
 - An unpruned tree ($\alpha = 0$).
 - Your optimally pruned tree.

Question: Describe the geometric shape of the boundaries. Why are there only horizontal and vertical lines?

```

1 tree_1 = DecisionTreeClassifier(ccp_alpha=0, random_state=42).fit(X_train, y_train
2   )
3 tree_2 = DecisionTreeClassifier(ccp_alpha=opt_alpha, random_state=42).fit(X_train,
4   y_train)
5
6 fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(16, 6))
7 plot_surface(tree_1, ax1, "plot1")
8 plot_surface(tree_2, ax2, "plot2")
9 plt.show()

```

Listing 2: 2-panel plot